

Lucrarea nr. 3

1. Scopul lucrării

Lucrarea are ca scop studiul semnalelor în domeniul spectral. Se vor reprezenta secvențe numerice folosind seria Fourier şi transformata Fourier discretă. De asemenea, vor fi prezentați algoritmi rapizi de calcul al transformatei Fourier, ce pot fi aplicați pe procesoare digitale de semnal.

2. Noțiuni teoretice

În mediul de programare MATLAB, calculul transformatei Fourier se realizează cu ajutorul următoarelor funcții:

X = *fft*(x)	**Calculează coeficienții transformatei Fourier discrete**
X = *ifft*(x)	**Calculează inversa transformatei Fourier discrete**
x – reprezintă secvența de intrare	
X – reprezintă coeficienții transformatei Fourier discrete	

Calculul **seriei Fourier discrete** se realizează după următoarele relații:

$$c_k = \frac{1}{N}\sum_{n=0}^{N-1}\tilde{x}[n]\cdot e^{-jk\frac{2\pi}{N}n}$$ Analiza secvenței $\tilde{x}[n]$

$$\tilde{x}[n] = \sum_{k=0}^{N-1}c_k\cdot e^{+jk\frac{2\pi}{N}n}$$ Sinteza secvenței $\tilde{x}[n]$

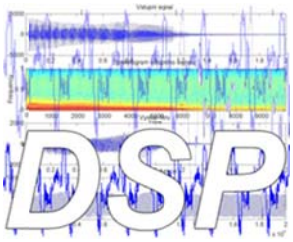
unde $\tilde{x}[n]$ este o secvență reală periodică.

Transformata Fourier discretă (TFD)

$$X[k]=\sum_{n=0}^{N-1}x[n]e^{-j2\pi kn/N}=\sum_{n=0}^{N-1}x[n]W_N^{kn}$$

Transformata Fourier discretă inversă (TFDI)

$$x[n]=\frac{1}{N}\sum_{k=0}^{N-1}X[k]e^{j2\pi kn/N}=\frac{1}{N}\sum_{k=0}^{N-1}X[k]W_N^{-kn}$$



Reprezentarea matriceală este $X = Wx \leftrightarrow x = \frac{1}{N} W^* X$, unde $W_N^{nk} = e^{-j2\pi kn/N}$.

Coeficienții TFD $\{X[k]\}$ sunt variabile complexe caracterizate de modul și fază:

$$X[k] = |X(k)| e^{j\phi(k)}$$

unde amplitudinea spectrală este:

$$|X(k)| = \sqrt{\{\mathrm{Re}\{X[k]\}\}^2 + \{\mathrm{Im}\{X[k]\}\}^2}$$

iar spectrul fazei este:

$$\phi[k] = \arctan \frac{\mathrm{Im}\{X[k]\}}{\mathrm{Re}\{X[k]\}}$$

Aplicațiile TFD necesită operații cu un volum mare de date, deoarece mărea numărului N al datelor, precizia de calcul crește. Rezultă că volumul de calcule crește în mod corespunzător. Astfel, pentru determinarea unui coeficient $X[k]$ al TFD sunt necesare *N* înmulțiri și *N-1* adunări în complex. Determinarea tuturor coeficienților ($k = 0,1,...,N-1$) implică N^2 înmulțiri și $N(N-1)$ adunări în complex. În aplicații, N ia valori de ordinul, cel puțin, $N = 2^{10} = 1024$, ceea ce conduce la valori inacceptabile pentru N^2.

În vederea reducerii volumului de calcule s-au elaborat metode prin care secvența de *N* valori x_n este împărțită în subsecvențe tot mai mici, în care operațiile de calcul sunt de același tip, datorită simetriei și periodicității exponențialelor w_N^{nk}. *A* fost elaborată o mare varietate de algoritmi de calcul a TFD[1], cunoscuți sub denumirea *transformata Fourier rapidă (TFR) – fast Fourier transform (FFT).*

Diferitele variante au performanțe comparabile și operează în general cu $N = 2^m$, ceea ce permite divizarea continuă cu 2, a secvențelor de date.

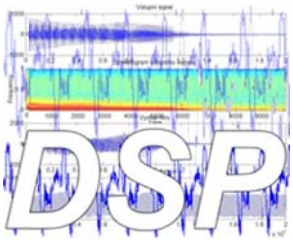
Există două clase de algoritmi pentru calculul TFD**:** cu decimare în timp (*DIT*) și cu decimare în frecvență.

2.1. TFR cu decimare în timp

Termenul de „*decimare*" în timp presupune *divizarea în subsecvențe mai scurte a secvenței* $\{x_n\}$*, care conține* $N = 2$ *eșantioane astfel încât secvența*

[1] Mateescu, Ad., ș.a., *Prelucrarea numerică a semnalelor*, Editura Tehnică, București, 1997





$X[k]$ *să rezulte din combinarea transformatelor corespunzătoare.* Acest lucru este posibil datorită următoarelor proprietăți ale secvenței $\{W_N^{kn}\}$, care se verifică imediat ținând seama că $W_N = e^{-j\frac{2\pi}{N}}$:

$$W_N^{2k} = W_{\frac{N}{2}}^{2k}$$

$$W_N^{k(N-n)} = \left(W_N^{kn}\right)^*$$

$$W_N^{kn} = W_N^{(k+N)n} = W_N^{k(N+n)}$$

Ultimele două egalități ilustrează *simetria*, respectiv *periodicitatea* secvențelor W_N^{kn}.

În cazul algoritmului TFR cu decimare în timp, secvența de intrare $\{x[n]\}$, $n = \overline{0, N-1}$ se împarte în două secvențe: pară $\{g[n]\}$ și impară $\{h[n]\}$, separând eșantioanele cu indice par de cele cu indice impar:

$$\left.\begin{aligned} g[n] &= x[2n] \\ h[n] &= x[2n+1] \end{aligned}\right\}, \quad n = \overline{0, \frac{N}{2}-1}$$

Separând secvența pară de cea impară în expresia pentru $X[k]$, se obține:

$$X[k] = \sum_{n=0}^{N/2-1}\left\{x[2n]W_N^{2nk} + x[2n+1]W_N^{(2n+1)k}\right\}, \qquad k = \overline{0,\ N-1}$$

Avem că:

$$W_N^{2nk} = e^{-j2\pi\cdot 2nk/N} = e^{-j\frac{2\pi}{N/2}nk} = W_{\frac{N}{2}}^{nk}$$

și rezultă:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n]W_{\frac{N}{2}}^{nk} + W_N^k \sum_{n=0}^{N/2-1} x[2n+1]W_{\frac{N}{2}}^{nk}, \qquad k = \overline{0,\ N-1}$$

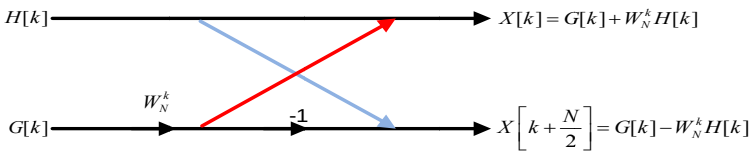
Se observă că acestea reprezintă TFD a secvenței pare, respectiv impară. Avem:

$$G[k] = \sum_{n=0}^{N/2-1} g[n]W_{\frac{N}{2}}^{nk}, \quad H[k] = W_N^k \sum_{n=0}^{N/2-1} h[n]W_{\frac{N}{2}}^{nk}, \quad k = \overline{0,\ \frac{N}{2}-1}$$

Secvențele $\{G[k]\}$ și $\{H[k]\}$ au perioada $\frac{N}{2}$, și prin urmare:

$$G[k] = G\left[k + \frac{N}{2}\right], \quad H[k] = H\left[k + \frac{N}{2}\right]$$

Descompunerea unei TFD de lungime *N*, în două TFD de lungime $\frac{N}{2}$, $N=8$, utilizând algoritmul de decimare în timp



Reprezentarea grafică a unei etape de divizare (structură *butterfly*)

Numărul total de înmulțiri în complex este acum $2\left(\frac{N}{2}\right)^2 + N = N^2\left[\frac{1}{2}+\frac{1}{N}\right] \cong \frac{N^2}{2}$ la N mare.

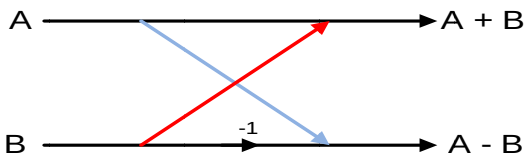
Relațiile de mai sus reprezintă esența algoritmului TFR cu decimare în timp, ilustrând un modul de combinații elementare, care reduce numărul de înmulțiri la jumătate. Acest modul este numit *etapă de divizare* sau *fluture* (*butterfly*). Factorul W_N^{nk} se numește *factor de rotație* (sau *coeficient de rotație*). Diviziunea cu 2 a secvențelor din domeniul timp este continuată. $\{G[k]\}$ și $\{H[k]\}$ se obțin din subsecvențe de lungime $\frac{N}{4}$. Decimarea în timp poate fi repetată de *m* ori, în ultima etapă de divizare cu doi obținându-se o TFD cu *N=2,* care necesită o adunare și o scădere, deoarece aici factorul de rotație este $W_N^{\frac{N}{2}} = e^{-j\pi} = -1$. În

total, pentru determinarea secvenței $\{X[k]\}$ sunt necesare următoarele operații în complex:

$$\frac{N}{2} \times m = \frac{N}{2} \log_2 N \text{ înmulțiri și } N \times m = N \log_2 N \text{ adunări.}$$

Reducerea volumului de calcule este spectaculoasă pentru N mare. De exemplu, pentru $N = 2^{10} = 1024$, rezultă o reducere ceva mai mare de 100 de ori.

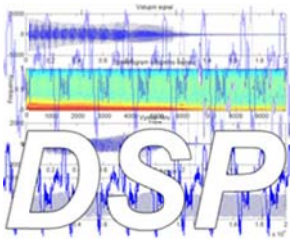
Graful TFR cu decimare în timp pentru $N = 8$



Graful pentru TFR cu $N = 2$

Sunt evidente câteva caracteristici generale ale TFR cu decimare în timp:

- ordinea datelor de intrare $\{x[n]\}$, $n = \overline{0, N-1}$ nu este cea naturală, în timp ce datele de ieșire $\{X[k]\}$ apar în ordine naturală. Ordinea intrărilor nu este întâmplătoare, ci urmează „*legea de inversiune binară*”, în care ordonarea se face „*cu inversarea biților*”. În această ordonare poziția *n,* ocupată de eșantionul de intrare cu indicele *n*, are în cod binar numărul *n* reprezentat în ordinea inversată a biților. În tabelul următor se ilustrează inversarea biților în cazul $N = 8$.



Ordinea eșantioanelor la intrare pentru TFR cu decimare în timp, *N*=8

Indexul eșantionului de intrare		Indexul eșantionului cu inversarea biților	
Zecimal	Binar	Binar	Zecimal
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

2.2. TFR cu decimare în frecvență

Algoritmul TFR cu decimare în frecvență este echivalent din punct de vedere al volumului de calcule cu cel precedent. Secvența de intrare $\{x[n]\}$ a datelor de intrare se împarte în două subsecvențe în modul următor:

$$\left.\begin{aligned} x[n]^1 &= x[n] \\ x[n]^2 &= x\left[n+\frac{N}{2}\right] \end{aligned}\right\}, \quad n=\overline{0,\ \frac{N}{2}-1}$$

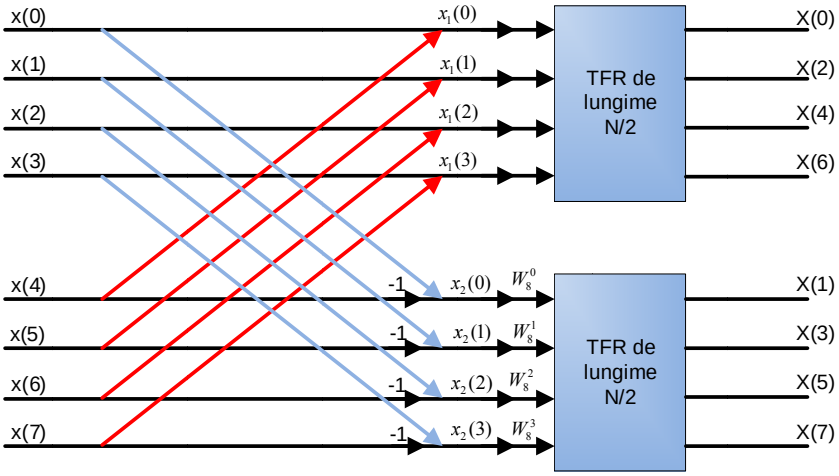
Relația pentru **TFR**, devine:

$$X[k]=\sum_{n=0}^{N/2-1} x[n]W_N^{nk}+\sum_{n=\frac{N}{2}}^{N/2-1} x[n]W_N^{nk}, \qquad k=\overline{0,\ N-1}$$

$$X[k]=\sum_{n=0}^{N/2-1} x[n]^1 W_N^{nk}+\sum_{n=0}^{N/2-1} x[n]^2 W_N^{\left(n+\frac{N}{2}\right)k}, \qquad k=\overline{0,\ N-1}$$

Se realizează acum decimarea în frecvență, separând eșantioanele $X[k]$ de ordin par, $k=2r$, de cele de ordin impar, $k=2r+1$:

$$\left.\begin{aligned} X[2r] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 W_{\frac{N}{2}}^{nr} + x[n]^2 W_{\frac{N}{2}}^{nr} \right] \\ X[2r+1] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 W_{\frac{N}{2}}^{nr} + x[n]^2 W_{\frac{N}{2}}^{\left(nr+\frac{N}{2}\right)} \right] \end{aligned}\right\} \quad r = \overline{0,\ \frac{N}{2}-1}$$

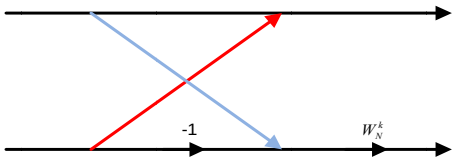


Descompunerea unei TFD de lungime *N*, în două TFD de lungime $\frac{N}{2}$, $N=8$, utilizând algoritmul de decimare în frecvență

Relațiile precedente pot fi simplificate sub forma:

$$\left.\begin{aligned} X[2r] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 + x[n]^2 \right] W_{\frac{N}{2}}^{nr} \\ X[2r+1] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 - x[n]^2 \right] W_N^r W_{\frac{N}{2}}^{nr} \end{aligned}\right\} \quad r = \overline{0,\ \frac{N}{2}-1}$$

Se obțin astfel TFD ale unor secvențe de lungime $\frac{N}{2}$.



***Fluture* (*butterfly*) pentru TFD cu decimare în frecvență**

3. Aplicații

1. Să se realizeze descompunerea în *SFTD* a unei serii de impulsuri periodice cu factorul de umplere 0,5. Se vor folosi 64 de eșantioane. Să se verifice apoi proprietățile generale ale coeficienților descompunerii în *SFTD* a unei secvențe periodice, cu valori reale, pentru care:

$$c(1)=\frac{1}{N}\sum_{n=0}^{N-1}x[n+1] \qquad c\left(\frac{N}{2}\right)=\frac{1}{N}\sum_{n=0}^{N-1}x[n+1](-1)^n \qquad c_{N-k}=c_k^*$$

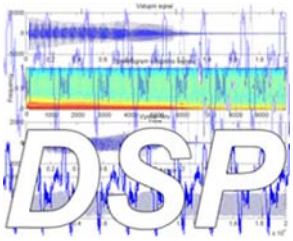
Programul MATLAB:

```
x=[ones(1,32),zeros(1,32)];N=64;
for k=0:N-1
    c(k+1)=0;
    for n=0:N-1
        c(k+1)=c(k+1)+x(n+1)*...
exp(-j*pi*2*k/N*n);
    end
    c(k+1)=c(k+1)/N
end
stem(abs(c(1:N/2))); grid
```

2. Se realizează secvența MATLAB pentru simularea algoritmului TFR cu decimare în frecvență și compararea rezultatului obținut cu acest algoritm, cu cel obținut utilizând funcția *fft* din MATLAB.

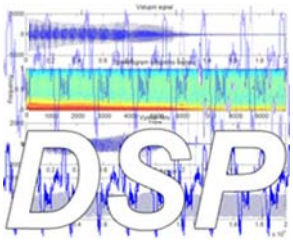
```
%simulare RFT
clear, clc;
%generarea semnalului pentru care se calculeaza spectru
%frecventa de esantionare
fe=10e3;
%lungime RFT
N=256;
t=0:1/fe:(N-1)/fe;
%frecventa sinusoida
fs=1e3;
n=0:length(t)-1;
```






```
x=sin(2*pi*fs/fe*n);
%reprezentarea in timp a semnalului
subplot(311);
plot(t(1:100),x(1:100));grid;
title('Reprezentarea semnalului in timp')
xlabel('Timp [s]');
ylabel('Amplitudine')
a=fft(x,N);
%reprezentare spectru semnal
subplot(312);
plot(abs(a));grid;
title('Reprezentarea spectrului semnalului(calculat cu
functia "fft" din Matlab)');
xlabel('Numar esantioane');
ylabel('Amplitudine');
%algoritm pentru calculul TFR utilizand decimarea in
frecventa
offset = N;
xi=x;
etape=0;
% for pentru numarul de etape
for et=1:(log2(N));
    offset = offset/2;
    nr_grupuri = N/(2*offset);
    x = xi;
    m=-1;
    etape=etape+1;
    % for pentru numarul de grupuri dintr-o etapa
    for l=1:nr_grupuri
        k=0;
        m=m+2;
        aux=((m-1)*offset)+1;
        %for pentru numarul de butterfly dintr-un grup
        for j=aux:((offset*m))
            xi(j) = (x(j)+x(j+offset));
            xa = (x(j)-x(j+offset));
            w(k+1) = exp((-i*2*pi*k)/N);
            xi(j+offset) = xa*w(k+1);
            k=k+2^(etape-1);
        end
    end
end
%ordonare esantioane la iesire
y=xi;
poz = 0:N-1;
for i=1:N
    poz_bin{i} = dec2bin(poz(i), log2(N));
end
for j=1:N
```





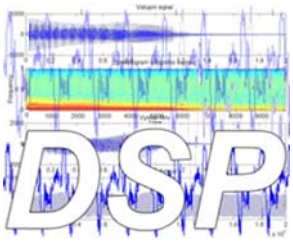

```
    for i=1:log2(N)
        poz_bin_ordonat{j}(i)=poz_bin{j}(log2(N)+1-i);
    end
end
poz_ordonat = bin2dec(poz_bin_ordonat);
for i=1:N
    y(poz_ordonat(i)+1)=xi(i);
end
%repreyentare grafica
subplot(313);
plot(abs(y));grid;
title('Reprezentarea spectrului semnalului (calculat
utilizand algoritmul de decimare in frecventa)');
xlabel('Numar esantioane');
ylabel('Amplitudine');
```

3. Secvența MATLAB următoare analizează în domeniile timp și frecvență un semnal *chirp*, a cărui frecvență instantanee variază între 0 și $5MHz$, pe o durată $T = 10\mu s$. Frecvența de eșantionare utilizată este $F_e = 50MHz$.

```
f0=0;ff=5e6; T=1e-5; beta=(ff-f0)*pi/T;
Fe=5e7;t=0:1/Fe:T;
x=cos(2*pi*f0*t+beta*t.^2);
subplot(211),plot(t,x),
xlabel('timp [s]'); ylabel('amplitudine [V]');
title('semnal temporal')
X=abs(fft(x,1024));
subplot(212), plot([0:99]/Fe,X(1:100))
xlabel('frecventa [Hz]'); ylabel('amplitudine [V]');
title('spectrul semnalului')
```

4. Să se elaboreze o funcție MATLAB pentru calculul TFTD a unei secvențe finite, de lungime N, pentru N frecvențe echidistante pe cercul unitate. Să se utilizeze funcția creată pentru calculul TFTD a secvenței $x[n] = 0,88^n$ în $N = 128$ puncte.

```
function [H,W]=tftd(h,N)
W=(2*pi/N)*[0:N-1]'; mid=ceil(N/2)+1;
W(mid:N)=W(mid:N)-2*pi; W=fftshift(W);
H=fftshift(fft(h,N));
subplot(211),plot(W/2/pi,abs(H));grid;
xlabel('frecventa normalizata')
ylabel('amplitudine [V]')
subplot(212),plot(W/2/pi,180/pi*angle(H));grid;
xlabel('frecventa normalizata')
ylabel('faza [grade]'); end
```



5. Reprezentați densitatea spectrală de putere a unui semnal format din două sinusoide de $50Hz$ și $120Hz$ și un zgomot aditiv, alb, gaussian, de medie nulă și dispersie 4.

```
t=0:0.001:0.8;
x=sin(2*pi*50*t)+sin(2*pi*120*t)+2*randn(1,length(t));
subplot(211);plot(x);
title('Semnalul x(t)');
grid;
X=fft(x,512);
Px=X.*conj(X)/512;
f=1000*(0:255)/512;
subplot(212);plot(f,Px(1:256));
title('Densitatea spectrala de putere');
grid
```

6. Să se verifice proprietățile de translație (deplasare, decalaj) în timp și de modulație ale transformatei Fourier. Se va considera semnalul:

$$x[n]=\begin{cases} n, n\in[1,8] \\ 0, n\in[9,16] \end{cases}$$

```
figure
w = -pi:2*pi/255:pi;
wo = 0.4*pi; D = 10;
num = [1 2 3 4 5 6 7 8 9];

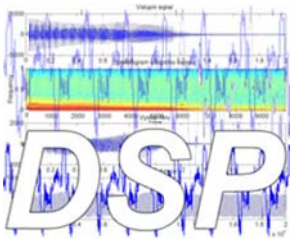
h1 = freqz(num, 1, w);
h2 = freqz([zeros(1,D) num], 1, w);

subplot(2,2,1)
plot(w/(2*pi),abs(h1));grid
title('Spectrul de amplitudine al secventei initiale')
subplot(2,2,2)
plot(w/(2*pi),abs(h2));grid
title('Spectrul de amplitudine al secventei decalate')
subplot(2,2,3)
plot(w/(2*pi),angle(h1));grid
title('Spectrul de faza al secventei initiale')
subplot(2,2,4)
plot(w/(2*pi),angle(h2));grid
title('Spectrul de faza al secventei decalate')

figure

w = -pi:2*pi/255:pi; wo = 0.4*pi;
num1 = [1 3 5 7 9 11 13 15 17];
```






```
L = length(num1);
h1 = freqz(num1, 1, w);
n = 0:L-1;
num2 = exp(wo*1i*n).*num1;
h2 = freqz(num2, 1, w);

subplot(2,2,1)
plot(w/(2*pi),abs(h1));grid
title('Spectrul de amplitudine al secventei initiale')
subplot(2,2,2)
plot(w/(2*pi),abs(h2));grid
title('Spectrul de amplitudine al secventei modulate')
subplot(2,2,3)
plot(w/(2*pi),angle(h1));grid
title('Spectrul de faza al secventei initiale')
subplot(2,2,4)
plot(w/(2*pi),angle(h2));grid
title('Spectrul de faza al secventei modulate')
```

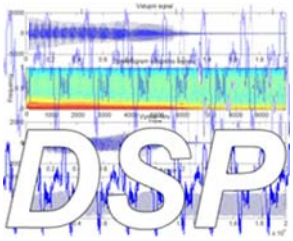
7. Să se studieze influența lungimii semnalului asupra spectrului său, estimat prin intermediul TFD. Se va considera un semnal sinusoidal de frecvență 10*Hz*, eșantionat la 100*Hz*. Să se studieze apoi efectul procedeului de *zero-paddding* și cazurile în care numărul de perioade este întreg sau nu.

```
figure(1)
t = 0:0.01:0.5-0.01;
x = cos(20*pi*t);
N = length(x) ; X = fft(x,N);
fp = (0:N-1)/N/0.01; fp = fp - 1/2/0.01;
stem(fp,fftshift(abs(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)')
ylabel('spectru de amplitudine')

figure(2)
X = fft(x,20*N); N = length(X);
fp = (0:N-1)/N/0.01;
fp = fp - 1/2/0.01;
plot(fp,abs(fftshift(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)')
ylabel('spectru de amplitudine')

figure(3)
t = 0:0.01:0.5-0.01-1/20;
x = cos(20*pi*t); N = length(x);
X = fft(x,N);
fp = (0:N-1)/N/0.01;
```






```
fp = fp - 1/2/0.01;
plot(fp,abs(fftshift(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)'); ylabel('spectru de
amplitudine')
```

Influența tipului ferestrei asupra spectrului estimat cu TFD

a) Să se genereze o sinusoidă de amplitudine $1V$, de frecvență $50Hz$, eșantionată la $256Hz$, în 32 de puncte. Să se calculeze TFD în 1024 de puncte considerând succesiv o fereastră dreptunghiulară, apoi triunghiulară, Hamming, Hanning și Blackman. Să se concluzioneze asupra rezoluțiilor spectrale și dinamice.

```
t=(1:32);f1=50;Fe=256;Nfft=1024 ;
y1=sin(2*pi*f1/Fe*t);sig=y1.*boxcar(32)' ;
y_rect=abs(fftshift((fft(sig,Nfft))));
sig=y1.*triang(32)';
y_tria=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hamming(32)' ;
y_hamm=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hanning(32)' ;
y_hann=abs(fftshift((fft(sig,Nfft))));
sig=y1.*blackman(32)';
y_blac=abs(fftshift((fft(sig,Nfft))));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
semilogy(f(513:1024),y_rect(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra dreptunghiulara',-1)
subplot(512);semilogy(f(513:1024),y_tria(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra triunghiulara',-1)
subplot(513);semilogy(f(513:1024),y_hamm(513:1024));
grid;ylabel('amplitudine spectrala')
axis([0 128 1e-4 100]);legend('fereastra Hamming',-1)
subplot(514);semilogy(f(513:1024),y_hann(513:1024));
axis([0 128 1e-3 100]);grid ;
legend('fereastra Hanning',-1)
subplot(515);semilogy(f(513:1024),y_blac(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra Blackman',-1)
xlabel('frecventa (Hz)')
```

b) Să se genereze un semnal compus din suma a două sinusoide de frecvențe $100Hz$ și $94Hz$ și de amplitudine $1V$. Se va considera aceeași lungime a semnalului de 32 puncte, iar TFD va fi calculată tot în 1024 de puncte.

```
t=(1:32);
f1=94;f2=100 ;
Fe=256;Nfft=1024 ;
y1= sin(2*pi*f1/Fe*t)+ sin(2*pi*f2/Fe*t);
sig=y1.*boxcar(32)' ;
y_rect=abs(fftshift((fft(sig,Nfft))));
sig=y1.*triang(32)';
y_tria=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hanning(32)' ;
y_hann=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hamming(32)' ;
y_hamm=abs(fftshift((fft(sig,Nfft))));
sig=y1.*blackman(32)';
y_blac=abs(fftshift((fft(sig,Nfft))));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
plot(f(513:1024),y_rect(513:1024));
grid;axis([0 128 0 20])
legend('fereastra dreptunghiulara',-1)
subplot(512);
plot(f(513:1024),y_tria(513:1024));grid;
axis([0 128 0 7])
legend('fereastra triunghiulara',-1)
subplot(513);
plot(f(513:1024),y_hann(513:1024));
grid;
axis([0 128 0 7])
legend('fereastra Hanning   ',-1)
ylabel('amplitudine spectrala')
subplot(514);
plot(f(513:1024),y_hamm(513:1024));
grid;
axis([0 128 0 7])
legend('fereastra Hamming ',-1)
subplot(515);
plot(f(513:1024),y_blac(513:1024));
grid;
axis([0 128 0 6])
legend('fereastra Blackman',-1)
xlabel('frecventa (Hz)')
```

c) Să se genereze semnalul următor în 32 de puncte:

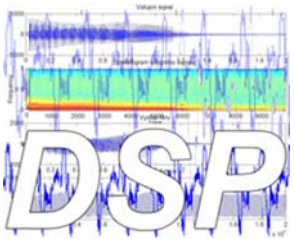
$$x[n] = \sin\left(2\pi \frac{100}{256} n\right) + 0{,}1\sin\left(2\pi \frac{74}{256} n\right)$$

Reprezentați spectrul său utilizând diverse ponderări şi interpretați rezultatele.

```
t=(1:32);f1=74;f2=100 ;Fe=256;Nfft=1024 ;
y1= 0.1*sin(2*pi*f1/Fe*t)+ sin(2*pi*f2/Fe*t);
sig=y1.*boxcar(32)';
y_rect=abs(fftshift(fft(sig,Nfft)));
sig=y1.*triang(32)';
y_tria=abs(fftshift(fft(sig,Nfft)));
sig=y1.*hanning(32)';
y_hann=abs(fftshift(fft(sig,Nfft)));
sig=y1.*hamming(32)';
y_hamm=abs(fftshift(fft(sig,Nfft)));
sig=y1.*blackman(32)';
y_blac=abs(fftshift(fft(sig,Nfft)));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
plot(f(513:1024),y_rect(513:1024));grid;axis([0 128 0
20])
legend('fereastra dreptunghiulara',-1)
subplot(512);plot(f(513:1024),y_tria(513:1024));grid;
axis([0 128 0 10])
legend('fereastra triunghiulara',-1)
subplot(513);plot(f(513:1024),y_hann(513:1024));grid;
axis([0 128 0 10])
legend('fereastra Hanning',-1)
ylabel('amplitudine spectrala')
subplot(514);plot(f(513:1024),y_hamm(513:1024));grid;
axis([0 128 0 10])
legend('fereastra Hamming ',-1)
subplot(515);plot(f(513:1024),y_blac(513:1024));grid;
axis([0 128 0 8])
legend('fereastra Blackman',-1); xlabel('frecventa
(Hz)')
```

8. Să se calculeze convoluția ciclică a două secvențe, x și h, de aceeași perioadă N, utilizând TFD_{1D}.

```
x=[1,2]; h=[3,4];
X=fft(x);H=fft(h);Y=X.*H;
y=abs(ifft(Y))
```



Analiza în domeniul frecvență a unui semnal folosind microcontrolerul dsPIC33FJ256GP506

9. Să se determine spectrul unui semnal audio folosind microcontrolerul dsPIC33FJ256GP506, de pe placa de dezvoltare *DSC Starter Kit*.

Semnalul audio analizat este inclus în codul programului, sub forma unui vector ce conține valorile eșantioanelor semnalului. Acestea au fost generate folosind programul *dsPICworks v*2.0. Semnalul este sinusoidal, de frecvență *1kHz*, eșantionat cu frecvența de 8*kHz*.

Algoritmul pentru determinarea spectrului unui semnal implică următorii pași:

1. Inițializarea coeficienților folosiți în algoritmul TFR;
2. Scalarea valorilor eșantioanelor în intervalul $[-0,5;0,5)$;
3. Calculul transformatei Fourier;
4. Inversarea biților rezultatelor obținute, deoarece rutina folosită la pasul anterior furnizează rezultatul cu biții în ordine inversă;
5. Calculul modulului numerelor complexe obținute anterior.

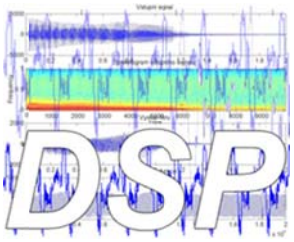
MPLAB cuprinde rutine de program scrise în limbaj de asamblare și în limbaj C, prin care se poate calcula spectrul unui semnal parcurgând pașii prezentați. Funcțiile folosite sunt:

- `extern fractcomplex* TwidFactorInit (int log2N, fractcomplex* twidFactors, int conjFlag);`

Inițializează coeficienții de rotație folosiți în cadrul algoritmului TFR. Aceștia sunt calculați ca numere reale, după care se realizează conversia lor în numere fracționare. Acceptă ca argumente $\log_2 N$, unde *N* reprezintă numărul de puncte în care se calculează transformata Fourier, pointer-ul către vectorul factorilor de rotație (**twidFactors* – *twiddle factors*) și un indicator care specifică dacă dorim generarea conjugatelor complexe ale coeficienților (*conjFlag*; 0 semnifică se dorește acest lucru).

- `extern fractcomplex* FFTComplexIP (int log2N, fractcomplex* srcCV, fractcomplex* twidFactors, int factPage);`

Calculează transformata Fourier a unei secvențe, înlocuind valorile inițiale cu cele obținute prin calcul (*IP – In Place*). Astfel, se utilizează mai eficient memoria disponibilă. Acceptă ca argumente $\log_2 N$, unde *N* reprezintă numărul de puncte în care se calculează transformata Fourier, vectorul complex pentru care se calculează transformata Fourier (*srcCV – source complex vector*), pointer-ul către vectorul factorilor de rotație (practic, adresa primului element din vector) și o variabilă care indică locația în care sunt stocați coeficienții de rotație, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*)



sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceştia). Vectorul returnat de această funcție conține componentele spectrale, având valorile în format fracționar complex și cu biții inversați.

- `extern fractcomplex* BitReverseComplex (int log2N, fractcomplex* srcCV) ;`

  Efectuează operația de inversare a biților unui număr fracționar complex. Acceptă ca argumente $\log_2 N$, unde N reprezintă numărul de puncte în care se calculează transformata Fourier, și pointer-ul către vectorul pentru care se aplică funcția (*srcCV*).

- `extern fractional* SquareMagnitudeCplx (int numelems, fractcomplex* srcV, fractional* dstV);`

  Calculează modulul numerelor complexe fracționare, acestea fiind elemente ale unui vector. Trebuie precizat vectorul pentru care se aplică funcția (*srcV – source vector*), dimensiunea sa (*numelems – number of elements*) și vectorul în care se stochează rezultatele, ca numere fracționare reale (*dstV – destination vector*).

- `extern fractional Float2Fract (float aVal);`

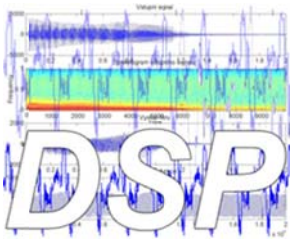
  Realizează conversia dintr-un număr real, de tip *float*, într-un număr fracționar, în gama $[-1,1)$.

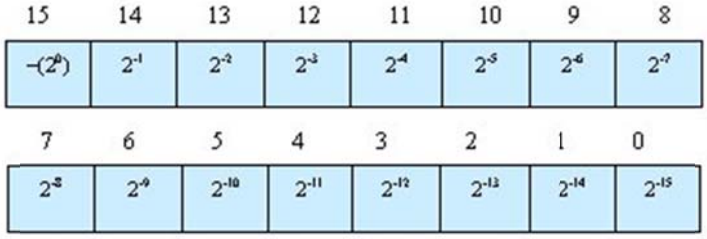
- `extern float Fract2Float (fractional aVal).`

  Realizează conversia dintr-un număr fracționar, în gama $[-1,1)$, într-un număr real, de tip *float*.

În general, algoritmii implementați pe procesoare de semnal sunt proiectați astfel încât să lucreze cu numere fracționare, care au gama de reprezentare $[-1,1)$. Unul din motivele pentru care se folosește acest format este că produsul a două numere fracționare este mai mic decât acestea[2]. În aplicația noastră, se folosește formatul *1.15*, care utilizează un bit de semn și 15 biți fracționari, așa cum se observă mai jos:

[2] Kester, W., *Mixed-signal and DSP design techniques*, Elsevier Science, 2003





De exemplu, numărul 0011 0000 0000 0000, sau 0x3000, în format fracționar, reprezintă valoarea 0,375 (0,25+0,125).

Precizia limitată a formatului *1.15* produce erori de cuantizare. De exemplu, cea mai apropiată reprezentare a numărului 0,2, în formatul *1.15*, este 0x199A, echivalentă însă cu 0,20001220703125, rezultând o eroare de cuantizare de $1,220703125 \cdot 10^{-5}$.

Scalarea valorilor eșantioanelor în intervalul $[-0,5;0,5)$ se face printr-o deplasare la dreapta cu un bit, adică o împărțire la doi.

Fișierele menționate mai sus vor fi copiate în proiectul creat, în directorul *src*. Directorul *h* va conține fișierele antet. Utilizarea celor două directoare este utilă la identificarea fișierelor în cazul în care proiectul cuprinde mai multe.

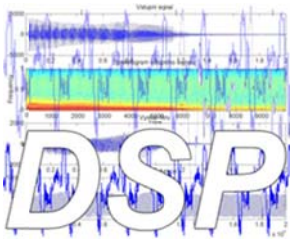
Așadar, în directorul *h* se vor copia:

- *dspcommon.inc* – conține declarații ale unor regiștri și configurări pentru calcule cu numere fracționare, fiind utilizat în rutinele de asamblare; se găsește în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\inc*;
- *dsp.h* – conține declarațiile funcțiilor folosite pentru algoritmul TFR; se găsește în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\include*.

În directorul *src* se vor copia:

- *bitrev.s* – rutina de inversare a biților;
- *cplxsqrmag.s* – rutina de calcul a modulului valorilor complexe;
- *fft.s* – rutina de calcul a transformatei Fourier;
- *flt2frct.c* – funcția de conversie din numere reale de tip *float* în numere fracționare;
- *frct2flt.c* – funcția de conversie din numere fracționare în numere reale de tip *float*;
- *initwid.c* – rutina de generare a coeficienților de rotație.

Toate aceste fișiere se găsesc în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*, respectiv *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\src*.



Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locația *D:\Projects\h*, iar în arborele de căutare pentru fișiere scrise în limbaj de asamblare (*Assembler Include Search Path*), locația *D:\Projects\src*.

Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define FRAME_SIZE 128
#define LOG2_BLOCK_LENGTH 7

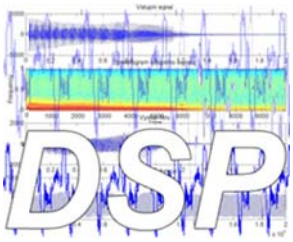
//Coeficientii de rotatie sunt salvati in memoria de date X
fractcomplex twiddleFactors   [FRAME_SIZE/2] __attribute__
((section (".xbss, bss, xmemory"), aligned (FRAME_SIZE*2)));
//Rezultatul TFR este salvat in memoria de date Y
fractcomplex sigCmpx          [FRAME_SIZE] __attribute__
((far)) _YBSS(FRAME_SIZE *2 *2);
fractional      magsig              [FRAME_SIZE/2];

//Programul principal
int main (){

        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01);        //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while (OSCCONbits.COSC != 0b01);      //Astepta
realizarea comutarii
        while(!OSCCONbits.LOCK);
```






```
        //Declararea variabilelor
        int samples[FRAME_SIZE] =
        {0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
        0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
        0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
        0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
        0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
        0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
        0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,
        0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,
        0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
        0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
        0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
        0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
        0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
        0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
        0x0000,0xA57F,0x8001,0xA57F};

        int j;
        float spectru [FRAME_SIZE/2]; //Jumatate din spectrul
calculat, pentru optimizarea memoriei folosite
        //Datorita simetriei, cealalta jumatate este in
oglinda
        float temp;

        //Initializari
        TwidFactorInit(LOG2_BLOCK_LENGTH,&twiddleFactors[0],0
); //Initializarea coeficientilor folositi in algoritmul TFR

        //Bucla de program
        while(1){
                //Scalarea datelor in intervalul [-0,5, 0,5)
                for (j=0; j<FRAME_SIZE; j++){  //Crearea unui
vector de numere complexe, cu partea imaginara nula
                        temp = (float)(samples[j]);
                        sigCmpx[j].real = Float2Fract(temp) >>
1;
                        sigCmpx[j].imag = 0x0000;
                }

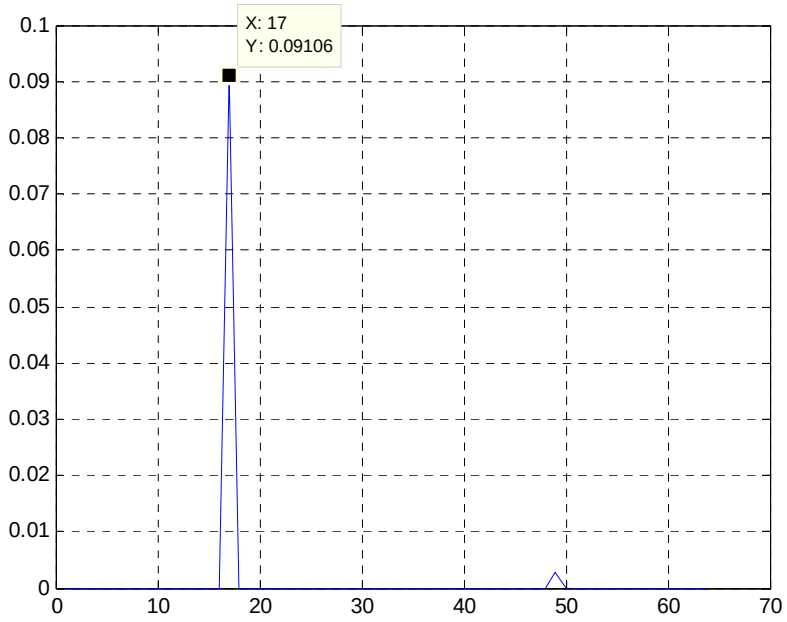
                FFTComplexIP
(LOG2_BLOCK_LENGTH,&sigCmpx[0],&twiddleFactors[0],COEFFS_IN_D
ATA);
                BitReverseComplex
(LOG2_BLOCK_LENGTH,&sigCmpx[0]);

        SquareMagnitudeCplx(FRAME_SIZE/2,&sigCmpx[0],&magsig[
0]); //magsig = |FFT|

                for (j=0;j<FRAME_SIZE/2;j++)
                        spectru[j] = Fract2Float(magsig[j]);
```

```
            while(1); //Ramane in bucla
        }
}
```

Spectrul obținut:



Frecvența de eșantionare este de $8kHz$, iar transformata Fourier a fost determinată în 128 de puncte. Rezultă un pas de:

$$\frac{8000}{128} = 62,5Hz$$

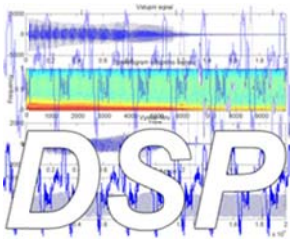
Deci, componenta spectrală determinată se află la frecvența:

$$62,5 \cdot 17 = 1062,5Hz$$

4. Desfășurarea lucrării

1. Se vor rula în MATLAB toate aplicațiile descrise in breviarul teoretic şi se va urmări execuția lor.
2. Să se genereze și să se reprezinte în timp și frecvență următoarele semnale:

- semnal sinusoidal cu frecvența $8kHz$ eșantionat la $30kHz$;
- semnalul format din suma a două semnale sinusoidale de $2kHz$, respectiv $3kHz$, eșantionate la $13kHz$;
- semnal dreptunghiular cu perioada de $125\mu s$, eșantionat la $15kHz$;



3. Să se realizeze modulația în amplitudine având semnalul modulator $x[n]=\sin(2\pi 0,01\cdot n)+\cos(2\pi 0,03\cdot n)$ și semnalul modulat $s[n]=\sin(2\pi 0,2\cdot n)$, unde $n\in N$ și $0\le n\le 300$. Gradul de modulație *m* este 0,6. Să se reprezinte semnalele din punct de vedere spectral.
4. Să se genereze și să se reprezinte în timp și frecvență un semnal *chirp*, a cărui frecvență instantanee variază între 10*kHz* și 10*MHz*, pe o durată $T=10\mu s$. Frecvența de eșantionare utilizată este $F_e=70MHz$.
5. Folosind funcția MATLAB pentru calculul TFTD prezentată în lucrare să se calculeze TFTD și să se reprezinte grafic modulul și faza pentru următoarele semnale:

- $x[n]=\sin(0,3n)$ în 64 de puncte;
- $x[n]=\log(0,2n)$ în 256 de puncte.